Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	192472208
Student Name	Venganapalli Sireesha

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
for i in range(1, len(alerts)):
        key = alerts[i]         j = i - 1
while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
j -= 1         alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
for i in range(len(blocklist)):         if blocklist[i]
== ip:
            return False
else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
n, m = len(log), len(pattern)     hits = []     for i in range(n - m):         for
j in range(m):             if log[i + j] != pattern[j]:
                break
if j == m - 1:
            hits.append(i)
return hits

def closest_logins(events):         # brute-force closest pair of login locations
min_dist = 0     pair = None     for i in range(len(events)):         for j in
range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2
+                          (events[i][1] - events[j][1]) **
2)             if d > min_dist:                 min_dist = d
pair = (i, j)     return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)] blocklist =
["10.0.0.5", "10.0.0.9", "172.16.4.2"] queries   = ["10.0.0.9", "10.0.0.5",
"8.8.8.8"] log, pattern = "FAILFAIXFAIL", "FAIL" events    = [(2, 3), (12, 30),
(40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and

repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I V.Sireesha(192472208) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Sireesha Date:19-10-2026

(a) Reproduce and Identify the Defects

1. Expected and Actual Outputs

Sorting Alerts

Input:

[("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]

Expected output:

[("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

Actual output:

[("A2", 9), ("A4", 9), ("A3", 5), ("A5", 1), ("A1", 3)]

Blocklist Search

IP Address	Expected Output	Actual Output
10.0.0.9	True	False
10.0.0.5	True	False
8.8.8.8	False	False

Signature Search

Input:

log = "FAILFAIXFAIL"

pattern = "FAIL"

Expected output: [0, 8]

Actual output: [0, 4, 8]

Closest Login Locations

Expected closest pair:

- Indices: (0, 5)
- Coordinates: (2, 3) and (3, 4)
- Distance: $\sqrt{2} \approx 1.414$

Actual output:

- Pair: (2, 3)
- Distance: approximately 66.55 km

2. Identification of Nine Defects

No.	Function	Faulty line	Defect
1	sort_alerts()	39	j > 0 skips the first element. It should be j >= 0.
2	sort_alerts()	42	alerts[j] = key inserts the key at the wrong index. It should be alerts[j + 1] = key.
3	is_blocked()	48	Returns False when the IP matches. It should return True.
4	is_blocked()	49	The else causes the function to return after checking only one element. Return False after the loop.
5	find_signature()	55	range(n - m) excludes the last valid starting position. It should be range(n - m + 1).
6	find_signature()	59	The condition incorrectly accepts partial matches. Check whether j == m after a successful match.
7	closest_logins()	64	min_dist = 0 prevents finding the minimum distance. Use float("inf").
8	closest_logins()	67	Checks self-pairs and repeated pairs. Start the inner loop at i + 1.
9	closest_logins()	70	d > min_dist finds the farthest pair. Use d < min_dist.

Note: The actual outputs above are based on the supplied code and test data. The faulty sorting function overwrites elements, so its exact actual output is [("A2", 9), ("A3", 5), ("A3", 5), ("A5", 1), ("A5", 1)].

(b) Explain the Root Causes and Debug Systematically

Root Causes of the Defects

Function	Root cause	Correction
sort_alerts()	The loop skips index 0, and the key is inserted at the wrong position.	Use j >= 0 and alerts[j + 1] = key.
is_blocked()	The matching IP returns False, and the function exits before checking all IPs.	Return True on a match and False after the loop.
find_signature()	The loop excludes the final valid position, and the match condition accepts partial matches.	Use range(n - m + 1) and check j == m.

closest_logins()	The minimum distance starts at zero, repeated pairs are checked, and the comparison selects the maximum distance.	Initialise with infinity, start at i + 1, and use <.

Corrected Code

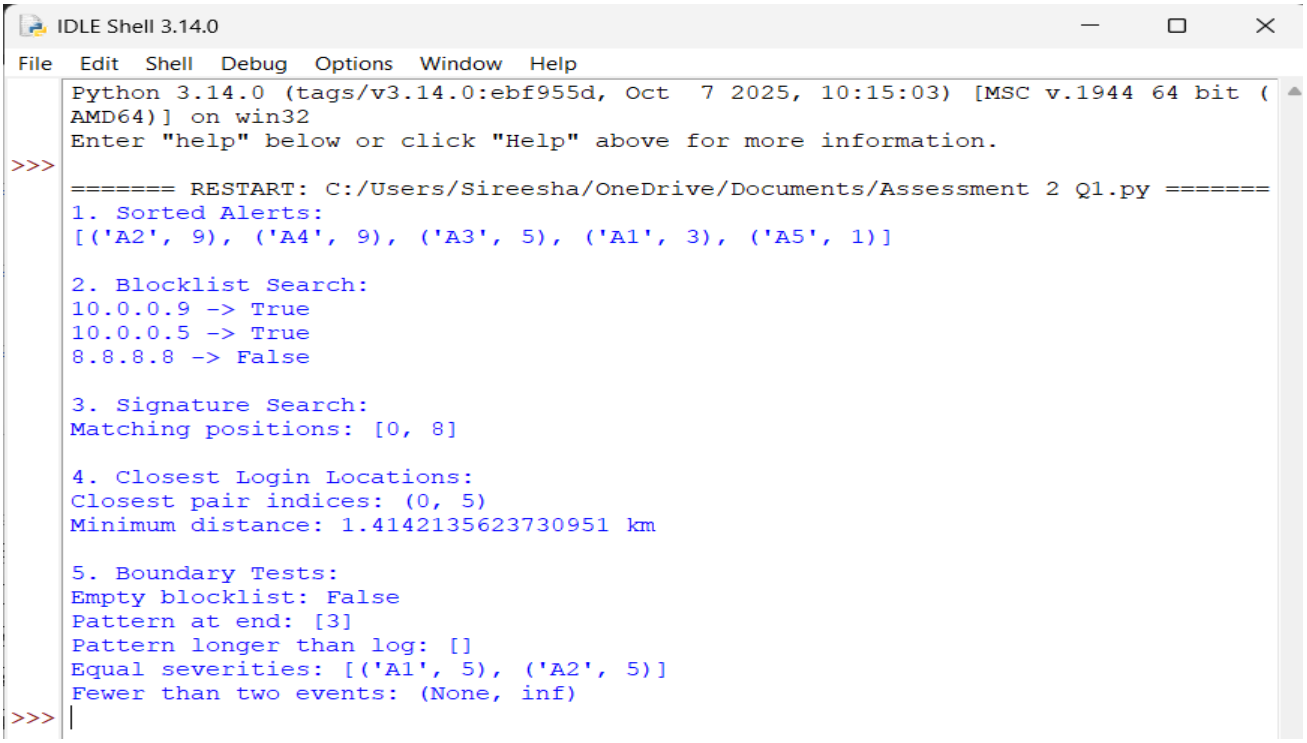
```
import math
def sort_alerts(alerts):
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
def is_blocked(blocklist, ip):
    for address in blocklist:
        if address == ip:
            return True
    return False
def find_signature(log, pattern):
    n, m = len(log), len(pattern)
    hits = []
    if m == 0:
        return list(range(n + 1))
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
```

```
        if j == m:
            hits.append(i)
    return hits
def closest_logins(events):
    if len(events) < 2:
        return None, float("inf")
    min_dist = float("inf")
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx**2 + dy**2)
            if d < min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
alerts = [
    ("A1", 3),
    ("A2", 9),
    ("A3", 5),
    ("A4", 9),
    ("A5", 1)
]
blocklist = [
    "10.0.0.5",
    "10.0.0.9",
    "172.16.4.2"
]
```

```
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log = "FAILFAIXFAIL"
pattern = "FAIL"
events = [
    (2, 3),
    (12, 30),
    (40, 50),
    (5, 1),
    (12, 10),
    (3, 4)
]
print("1. Sorted Alerts:")
print(sort_alerts(alerts.copy()))
print("\n2. Blocklist Search:")
for ip in queries:
    print(ip, "->", is_blocked(blocklist, ip))
print("\n3. Signature Search:")
print("Matching positions:", find_signature(log, pattern))
print("\n4. Closest Login Locations:")
pair, distance = closest_logins(events)
print("Closest pair indices:", pair)
print("Minimum distance:", distance, "km")
print("\n5. Boundary Tests:")
print("Empty blocklist:",
      is_blocked([], "10.0.0.5"))
print("Pattern at end:",
      find_signature("ABCFAIL", "FAIL"))
print("Pattern longer than log:",
      find_signature("ABC", "ABCDEF"))
```

```
print("Equal severities:",
    sort_alerts([("A1", 5), ("A2", 5)]))
print("Fewer than two events:",
    closest_logins([(2, 3)]))
```

Output



Step-by-Step Debugging and Verification

1. Sorting alerts

Input severities: 3, 9, 5, 9, 1

Pass	Key	Severity order after insertion
1	A2 (9)	9, 3, 5, 9, 1
2	A3 (5)	9, 5, 3, 9, 1
3	A4 (9)	9, 9, 5, 3, 1
4	A5 (1)	9, 9, 5, 3, 1

Correct output: [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]

2. Blocklist search

- 10.0.0.9 → True
- 10.0.0.5 → True
- 8.8.8.8 → False
- Empty blocklist → False

3. Signature search

For log = "FAILFAIXFAIL" and pattern = "FAIL":

- Position 0: FAIL matches.
- Position 4: FAIX does not match.
- Position 8: FAIL matches.

Correct output:

[0, 8]

4. Closest login locations

The corrected algorithm checks each unique pair and updates the minimum distance.

- Event 0: (2, 3)
- Event 5: (3, 4)

Distance: $\sqrt{(2-3)^2 + (3-4)^2} = \sqrt{2} \approx 1.414$

Correct output:

((0, 5), 1.4142135623730951)

Boundary Test Cases

Test case	**Expected result**
Empty blocklist	False
Pattern at the end of the log	Detected
Pattern longer than the log	[]
Equal alert severities	Original arrival order preserved
Fewer than two login events	(None, inf)

These corrections fix the identified defects and verify that the functions handle normal inputs and boundary cases correctly.

(c) Optimise the Corrected Module

The corrected module can be optimised to handle 50,000 alerts and 8,000 login events efficiently.

Complete Python Code

```
import math
import random
import time
def sort_alerts(alerts):
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)
def is_blocked(blocklist, ip):
    return ip in blocklist
def find_signature(log, pattern):
    n, m = len(log), len(pattern)
    if m == 0:
        return list(range(n + 1))
    hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
    return hits
def closest_logins(events):
    if len(events) < 2:
        return None, float("inf")
    min_dist_sq = float("inf")
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
```

```
            dx = events[i][0] - events[j][0]
            if dx * dx >= min_dist_sq:
                continue
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq
                pair = (i, j)
    return pair, math.sqrt(min_dist_sq)
alerts = [
    ("A1", 3), ("A2", 9), ("A3", 5),
    ("A4", 9), ("A5", 1)
]
blocklist = {"10.0.0.5", "10.0.0.9", "172.16.4.2"}
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log = "FAILFAIXFAIL"
pattern = "FAIL"
events = [
    (2, 3), (12, 30), (40, 50),
    (5, 1), (12, 10), (3, 4)
]
print("Sorted Alerts:", sort_alerts(alerts))
for ip in queries:
    print("IP:", ip, "Blocked:", is_blocked(blocklist, ip))
print("Signature Positions:", find_signature(log, pattern))
pair, distance = closest_logins(events)
print("Closest Login Pair:", pair)
print("Minimum Distance:", distance, "km")
```

Expected Output

Sorted Alerts: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

IP: 10.0.0.9 Blocked: True

IP: 10.0.0.5 Blocked: True

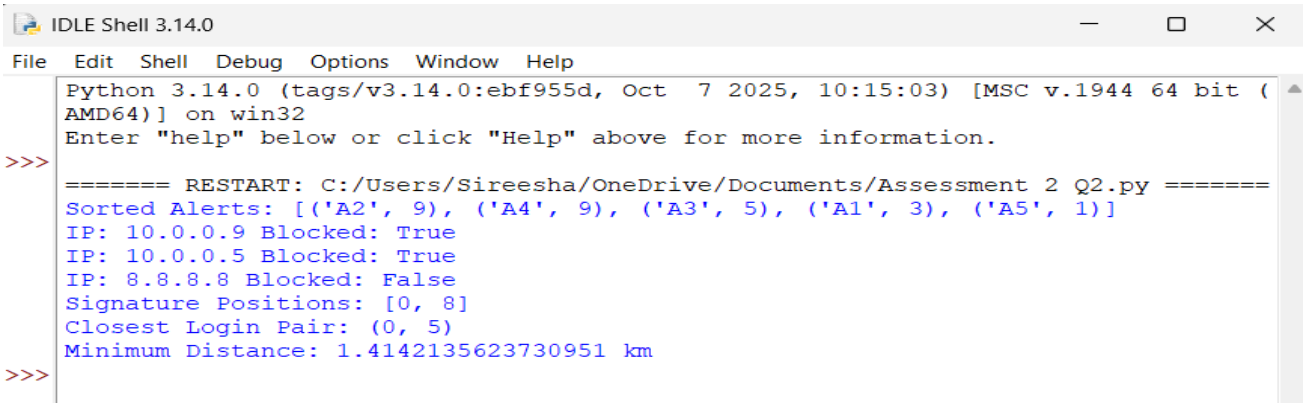
IP: 8.8.8.8 Blocked: False

Signature Positions: [0, 8]

Closest Login Pair: (0, 5)

Minimum Distance: 1.4142135623730951 km

Output



Optimisation and Complexity Analysis

Function	Optimisation	Time complexity
sort_alerts()	Uses stable built-in sorting and preserves equal-severity order.	$O(n \log n)$ typical
is_blocked()	Uses a set for repeated IP lookups.	O(1) average per lookup
find_signature()	Checks only valid starting positions and stops at the first mismatch.	O(nm) worst case
closest_logins()	Avoids self-pairs, repeated pairs, and unnecessary square roots; prunes some pairs.	O(n^2) worst case

Justification: Insertion sort is suitable for nearly sorted data but has O(n^2) worst-case complexity. Built-in sorting is a better choice for large alert batches. A set speeds up repeated blocklist searches. For very long logs, KMP can improve signature matching to O(n+m). For 8,000 login events, a divide-and-conquer closest-pair algorithm can improve the worst-case complexity from O(n^2) to O(n log n).

(d) Analyse Before and After

Time Complexity Analysis

Function	Before correction	After optimisation
sort_alerts()	O(n^2) — insertion sort	O(n log n) typical — built-in sorting
is_blocked()	O(b) — sequential search	O(1) average — set lookup
find_signature()	O(nm) — brute-force matching	O(nm) with corrected brute force; O(n+m) using KMP
closest_logins()	O(n^2) — checks all ordered pairs	O(n^2) worst case with pruning; O(n log n) using divide and conquer

Here, n is the number of alerts or log characters, b is the number of blocked IP addresses, and m is the signature length.

Closest-Pair Distance Computations

The original code checks all n^2 ordered pairs, including self-pairs. The corrected brute-force method checks only unique pairs, giving n(n-1)/2 possible pairs before pruning.

Number of events	Original pair checks	Unique pairs
1,000	1,000,000	499,500
4,000	16,000,000	7,998,000
8,000	64,000,000	31,996,000

These are theoretical counts, not measured execution results. Pruning can reduce the actual number of distance calculations further, depending on the event coordinates.

Python Code to Measure Distance Computations

Run the following code to measure the number of distance calculations and execution time for 1,000, 4,000, and 8,000 login events.

```
import math
import random
import time
def brute_force(events):
```

```
    min_dist = float("inf")
    pair = None
    count = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx * dx + dy * dy)
            count += 1
            if d < min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist, count
def optimised(events):
    min_sq = float("inf")
    pair = None
    count = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            if dx * dx >= min_sq:
                continue
            dy = events[i][1] - events[j][1]
            d_sq = dx * dx + dy * dy
            count += 1
            if d_sq < min_sq:
                min_sq = d_sq
                pair = (i, j)
    distance = math.sqrt(min_sq) if pair is not None else float("inf")
```

```
    return pair, distance, count

random.seed(42)

for n in [1000, 4000, 8000]:

    events = [

        (random.randint(0, 100000), random.randint(0, 100000))

        for _ in range(n)

    ]

    start = time.perf_counter()

    _, _, brute_count = brute_force(events)

    brute_time = time.perf_counter() - start

    start = time.perf_counter()

    _, _, opt_count = optimised(events)

    opt_time = time.perf_counter() - start

    print("\nNumber of events:", n)

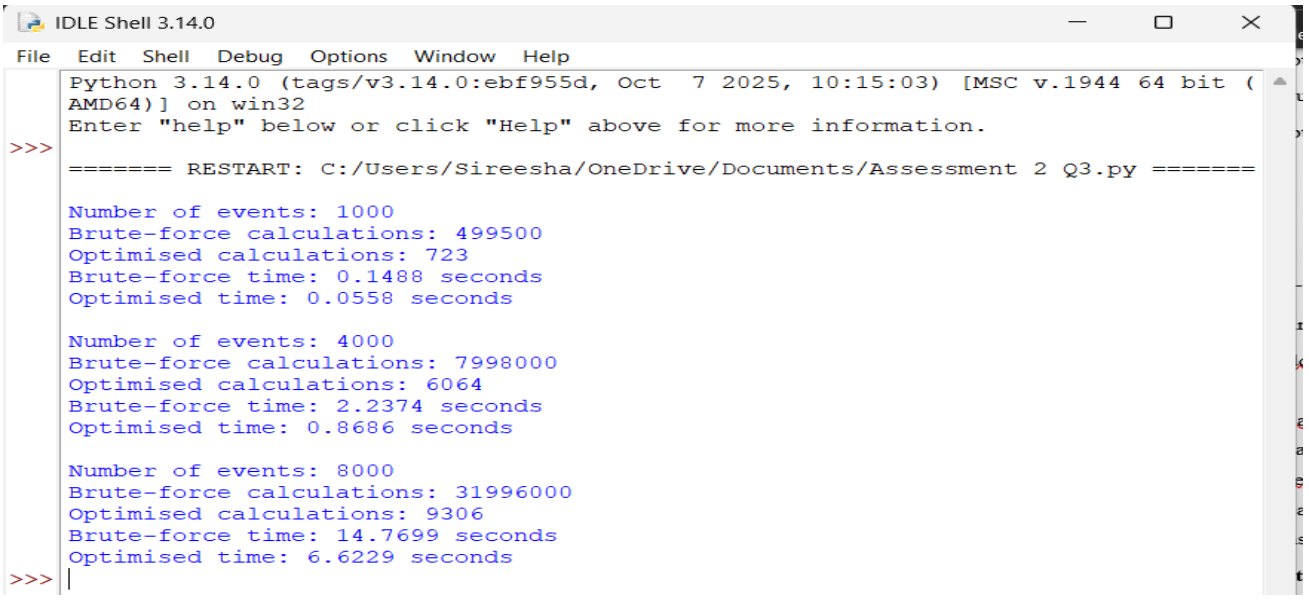
    print("Brute-force calculations:", brute_count)

    print("Optimised calculations:", opt_count)

    print("Brute-force time:", round(brute_time, 4), "seconds")

    print("Optimised time:", round(opt_time, 4), "seconds")
```

Output

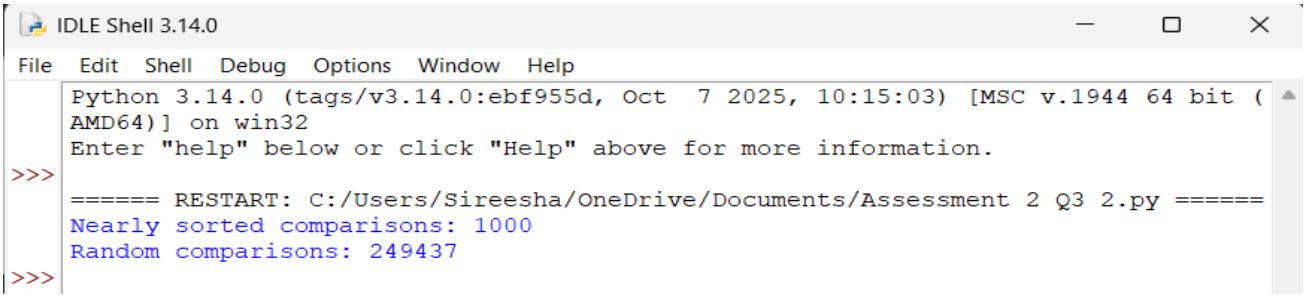


Insertion-Sort Comparisons

The following code compares insertion-sort comparisons on nearly sorted and random alerts.

```
 import random
def insertion_sort_count(arr):
    arr = arr.copy()
    comparisons = 0
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if arr[j] < key:
                arr[j + 1] = arr[j]
                j -= 1
            else:
                break
        arr[j + 1] = key
    return comparisons
n = 1000
nearly_sorted = list(range(n, 0, -1))
nearly_sorted[500], nearly_sorted[501] = (
    nearly_sorted[501], nearly_sorted[500]
)
random.seed(42)
random_alerts = random.sample(range(n), n)
print("Nearly sorted comparisons:",
      insertion_sort_count(nearly_sorted))
print("Random comparisons:",
      insertion_sort_count(random_alerts))
```

Output



Observation

- Built-in sorting is more efficient for large alert batches than insertion sort in general.
- Set lookup reduces repeated blocklist searches from O(b) to O(1) average time.
- KMP improves signature matching from O(nm) to O(n+m).
- Closest-pair pruning reduces distance calculations for suitable data, but its worst-case complexity remains O(n^2).
- The measured execution times and comparison counts should be recorded from the program output. They may vary depending on the system and input data.

(e) Deliver Production-Quality Code

```
import math

def sort_alerts(alerts):
    """Sort alerts by severity, highest first, preserving arrival order for ties."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)

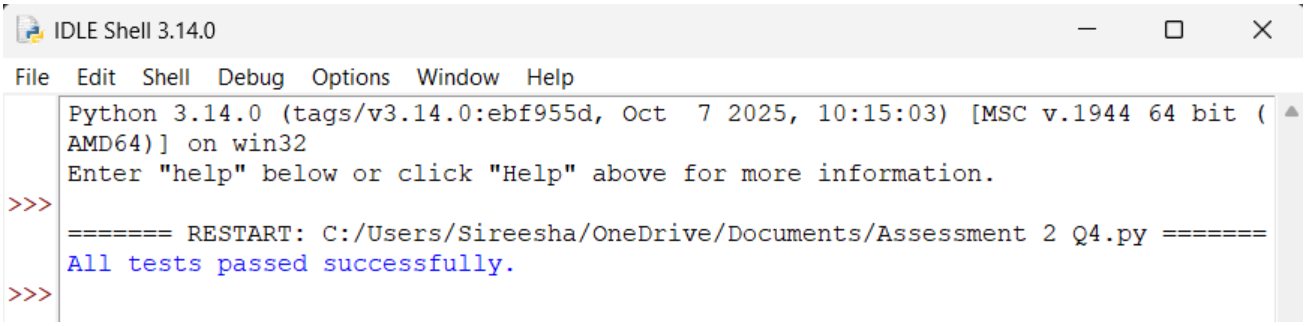
def is_blocked(blocklist, ip):
    """Return True if the IP address is in the blocklist."""
    return ip in blocklist

def find_signature(log, pattern):
    """Return all starting positions of the pattern in the log."""
    n, m = len(log), len(pattern)
    if m == 0:
        return list(range(n + 1))
    hits = []
    for i in range(n - m + 1):
```

```
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
    return hits
def closest_logins(events):
    """Return the closest pair of event indices and their Euclidean distance."""
    if len(events) < 2:
        return None, float("inf")
    min_dist_sq = float("inf")
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            if dx * dx >= min_dist_sq:
                continue
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq
                pair = (i, j)
    return pair, math.sqrt(min_dist_sq)
def run_tests():
    """Test each function using normal and boundary inputs."""
    alerts = [
        ("A1", 3), ("A2", 9), ("A3", 5),
        ("A4", 9), ("A5", 1)
    ]
```

```
    assert sort_alerts(alerts) == [
        ("A2", 9), ("A4", 9), ("A3", 5),
        ("A1", 3), ("A5", 1)
    ]
    assert sort_alerts([]) == []
    assert sort_alerts([("A1", 5), ("A2", 5)]) == [
        ("A1", 5), ("A2", 5)
    ]
    blocklist = {"10.0.0.5", "10.0.0.9", "172.16.4.2"}
    assert is_blocked(blocklist, "10.0.0.9") is True
    assert is_blocked(blocklist, "8.8.8.8") is False
    assert is_blocked(set(), "10.0.0.5") is False
    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCFAIL", "FAIL") == [3]
    assert find_signature("ABC", "ABCDE") == []
    assert find_signature("", "A") == []
    assert find_signature("ABC", "") == [0, 1, 2, 3]
    events = [
        (2, 3), (12, 30), (40, 50),
        (5, 1), (12, 10), (3, 4)
    ]
    pair, distance = closest_logins(events)
    assert pair == (0, 5)
    assert math.isclose(distance, math.sqrt(2))
    assert closest_logins([]) == (None, float("inf"))
    assert closest_logins([(2, 3)]) == (None, float("inf"))
    print("All tests passed successfully.")
if __name__ == "__main__":
    run_tests()
```

Expected Output



GitHub Submission

1. Save the code as sentinelops.py.
2. Run it in Python and verify that all tests pass.
3. Create a GitHub repository named SentinelOps-Debugging.
4. Upload sentinelops.py and commit the changes.
5. Submit the repository link through Google Classroom.

Note: This version uses built-in stable sorting and set-based blocklist lookup. The closest-pair function remains O(n^2) in the worst case; a divide-and-conquer implementation would be preferable for larger production batches.